> 📥 **Download:** Get the PDF version of these notes here

📊 SQL for Data Science: Quick Revision Notes

> A structured compilation of SQL concepts optimized for quick revision and interview prep. 🚀

📑 Table of Contents

1. SQL Basics
    2. SELECT Statement & Data Retrieval
2. SQL Useful Expressions (COUNT, DISTINCT, LIMIT)
3. INSERT Statement
4. UPDATE & DELETE Statements
5. Database Concepts
6. Types of SQL Statements
7. CREATE TABLE Statement
8. ALTER, DROP & TRUNCATE Statements
9. Advanced Filtering Techniques (LIKE, BETWEEN, IN)
10. Sorting Result Sets (ORDER BY)
11. Grouping and Filtering Result Sets
12. Built-in Database Functions
13. Date and Time Functions
14. Subqueries (Nested Queries)
15. Accessing Multiple Tables (Implicit Joins)

---

1. SQL Basics

SQL (Structured Query Language) is used to query or retrieve data from relational databases.

Core Concepts

- **Data:** A collection of facts (words, numbers, pictures). A critical asset for any business.
- **Database:** A program/repository to store, add, modify, and query data.
- **Relational Database:** Data organized in a tabular form (columns and rows, like a spreadsheet).
  - **Columns** contain item properties (e.g., `LastName`, `FirstName`).
  - Tables can have relationships with one another.

DBMS vs RDBMS

System	Description	Examples
DBMS	Database Management System. Tools to manage data within a database.	MongoDB, Redis
RDBMS	Relational Database Management System. Specifically for relational data.	MySQL, Oracle, DB2, PostgreSQL

5 Basic SQL Commands 🛠️

1. `CREATE` - To create a table.
2. `INSERT` - To populate/fill data into a table.
2. `SELECT` - To view/retrieve data.
3. `UPDATE` - To modify existing data.
4. `DELETE` - To remove data.

---

2. SELECT Statement & Data Retrieval

The `SELECT` statement is a **DML (Data Manipulation Language)** command used to read or retrieve data. It is often referred to as a query, and the output is called a *result set* or *result table*.

Basic Syntax

- **Retrieve all columns:**
  (Often read as "select star from table name")

```
SELECT *
FROM table_name;
```

- **Specific columns:**
  (Note: Columns are displayed in the exact order they are written in the query).

```
SELECT column1, column2 FROM table_name;
```

WHERE Clause & Predicate:

- The `WHERE` clause is used to filter and restrict the result set.
- **Predicate:** A condition that evaluates to true, false, or unknown.

```
SELECT book_id, title FROM book WHERE book_id = 'B1';
```

Comparison Operators (Used within the WHERE clause):

1. Equal to (`=`)
   2. Greater than (`>`)
2. Less than (`<`)
3. Greater than or equal to (`>=`)
4. Less than or equal to (`<=`)
5. Not equal to (`<>` or `!=`)

---

3. SQL Useful Expressions (COUNT, DISTINCT, LIMIT)

COUNT Function

- **Purpose:** Returns the total number of rows (count) that match the query criteria.
- **Examples:**
- Count all rows: `SELECT COUNT(*) FROM tablename;`

- Count with a condition: `SELECT COUNT(COUNTRY) FROM MEDALS WHERE COUNTRY = 'CANADA';`

DISTINCT Keyword

- **Purpose:** Removes duplicate values from the result set, displaying only unique values.
- **Examples:**
- Retrieve unique values: `SELECT DISTINCT columnname FROM tablename;`
- Unique countries that won gold: `SELECT DISTINCT COUNTRY FROM MEDALS WHERE MEDALTYPE = 'GOLD';`

LIMIT Clause

- **Purpose:** Restricts the number (quantity) of rows retrieved from the database.
- **Examples:**
- View the first 10 rows: `SELECT * FROM tablename LIMIT 10;`
- Apply limit on filtered data: `SELECT * FROM MEDALS WHERE YEAR = 2018 LIMIT 5;`

---

4. INSERT Statement

- **Purpose:** Used to add (populate) new rows of data into a table.
- **Type:** A DML (Data Manipulation Language) statement.

Basic Syntax:

```
INSERT INTO table_name (column1, column2, column3) VALUES (value1, value2, value3);
```

Key Rules:

1. **Column & Value Match:** The total count of values in the `VALUES` clause must be exactly equal to the number of columns listed.
   2. **Order Matters:** The values provided must strictly follow the same sequence as the specified columns.

Row Insertion Methods:

1. **One row at a time:** Inserting a single record.
2. **Multiple rows at a time:** Inserting multiple records in a single `INSERT` statement by separating value sets with a comma (`,`).

```
INSERT INTO author (author_id, last_name, first_name)
VALUES ('A1', 'Chong', 'Raul'),
       ('A2', 'Ahuja', 'Rav');
```

---

5. UPDATE & DELETE Statements

Type: Both are DML (Data Manipulation Language) statements used to modify or remove data.

1. UPDATE Statement

- **Purpose:** To alter or modify existing data within a table.
- **Basic Syntax:**

```
UPDATE table_name SET column1 = value1, column2 = value2 WHERE
condition;
```

- **Example:**

```
UPDATE AUTHOR SET LAST_NAME = 'KATTA', FIRST_NAME = 'LAKSHMI' WHERE
AUTHOR_ID = 'A2';
```

> ⚠️ **Critical Warning:** If you forget to include the `WHERE` clause, **all rows** in the table will be updated!

2. DELETE Statement

- **Purpose:** To permanently remove one or more rows from a table.

- **Basic Syntax:**

```
DELETE FROM table_name WHERE condition;
```

- **Example:**

```
DELETE FROM AUTHOR WHERE AUTHOR_ID IN ('A2', 'A3');
```

> ⚠ **Critical Warning:** If you execute `DELETE FROM table_name;` without a `WHERE` clause, the entire data of the table will be deleted (the table will be emptied)!

---

6. Database Concepts

1. Relational Model

- The most widely used data model.
- Stores data in a simple structure: Tables (Rows and Columns).
- **Main Advantage:** Provides Data Independence (Logical, Physical, and Physical storage independence).

2. ER Model (Entity-Relationship Model) & ERD

- Used as a conceptual tool to design relational databases.
- **ERD (Entity-Relationship Diagram):** A visual representation of entities and their relationships.
- **Building Blocks:**
- **Entity:** Any real-world object (Noun: Person, Place, Thing) that exists independently. (ERD Symbol: Rectangle). Examples: Book, Author, Borrower.
- **Attribute:** The characteristics or properties that provide details about the entity. (ERD Symbol: Oval). Examples: Title, Edition, and Year for a Book.

3. Mapping (ER Model to Relational Table)

ER Model Component	Relational Database Equivalent

Entity	Table
Attribute	Column
Data Values	Row / Tuple

4. Common Data Types

- **CHAR (Character):** For fixed-length text (e.g., ISBN, State codes).
- **VARCHAR (Variable Character):** For variable-length text (e.g., Title, Name, Email).
- **Numeric (INTEGER / DECIMAL):** For numbers (e.g., Edition, Year, Price).
- **Date / Time:** For storing dates and timestamps.

5. Database Keys

- **Primary Key (PK):** Uniquely identifies every single row (tuple) in a table. Prevents data duplication (no duplicates allowed, cannot be null).
- **Foreign Key (FK):** A Primary Key from another table that is added to the current table. Used to establish a relationship or link between tables.

---

7. Types of SQL Statements

SQL statements are used to interact with Entities (tables), Attributes (columns), and Tuples (rows/data values) in relational databases.

1. DDL (Data Definition Language)

- **Purpose:** To define, modify, or delete the structure of database objects (such as tables).
- **Common Statements:**
- `CREATE` : To create a new table and define its columns/data types.
- `ALTER` : To modify an existing table's structure.
- `TRUNCATE` : To delete all data inside a table while keeping the table structure intact.
- `DROP` : To permanently delete the entire table along with its structure.

2. DML (Data Manipulation Language)

- **Purpose:** To read and modify the data present inside tables. Often referred to as **CRUD** operations:

- **C**reate ➔ `INSERT`
- **R**ead ➔ `SELECT`
- **U**pdate ➔ `UPDATE`
- **D**elete ➔ `DELETE`

SQL Statement Categories Comparison

Category	Main Focus	Target	Common Commands
DDL	Defining the Structure / Schema	Database Objects (Tables)	CREATE, ALTER, TRUNCATE, DROP
DML	Manipulating the Data	Rows / Records	INSERT, SELECT, UPDATE, DELETE

---

8. CREATE TABLE Statement

Type: The most common DDL statement. Used to create new tables (Entities) and their columns (Attributes).

Basic Syntax Structure

```
CREATE TABLE table_name (
    column1_name data_type [constraints],
    column2_name data_type [constraints],
    column3_name data_type [constraints]
);
```

Column Components

1. **Column Name:** The name of the attribute (e.g., `author_id`, `firstname`).
2. **Data Type:** The format of the stored data:

- `CHAR(n)`: Fixed-length string.
- `VARCHAR(n)`: Variable-length string.

3. **Optional Constraints:** Rules applied for data validation.

Key Constraints Mentioned

- **PRIMARY KEY:** Uniquely identifies each row; does not allow duplicate records.
- **NOT NULL:** Ensures that a blank or NULL value cannot be inserted into the column.

Practical Example (Library Database)

```
CREATE TABLE author (
    author_id CHAR(2) PRIMARY KEY NOT NULL,
    lastname VARCHAR(15) NOT NULL,
    firstname VARCHAR(15) NOT NULL,
    email VARCHAR(40),
    city VARCHAR(15),
    country CHAR(2)
);
```

---

9. ALTER, DROP & TRUNCATE Statements

1. ALTER TABLE Statement

- **Purpose:** To modify the structure of an existing table.
- **Operations & Syntax:**
- **Add Column:**

```
ALTER TABLE author ADD COLUMN telephone_number BIGINT;
```

- **Modify Data Type:**

```
ALTER TABLE author MODIFY telephone_number CHAR(20);
```

> ⚠️ **Warning (Data Compatibility):** If the column already contains incompatible data, the query will fail.

- **Drop Column:**

```
ALTER TABLE author DROP COLUMN telephone_number;
```

2. DROP TABLE Statement

- **Purpose:** To permanently delete a table from the database (both structure and data).
- **Syntax Example:**

```
DROP TABLE author;
```

3. TRUNCATE TABLE Statement

- **Purpose:** To delete all rows (data) at once, while keeping the table structure intact. Much faster than a `DELETE` statement.
- **Syntax Example:**

```
TRUNCATE TABLE author IMMEDIATE;
```

Quick Comparison: DROP vs TRUNCATE vs DELETE

Command	Category	Table Structure Remains?	Data Deleted?	Speed
DELETE	DML	Yes	Yes (Selected or all rows)	Slow (Logs row-by-row)
TRUNCATE	DDL	Yes (Table becomes empty)	Yes (All rows)	Very Fast
DROP	DDL	No (Table is destroyed)	Yes (All rows)	Instant

---

10. Advanced Filtering Techniques (LIKE, BETWEEN, IN)

When we don't know the exact value for the `WHERE` clause, or when the query is getting too long, these three advanced filtering techniques are commonly used in the industry to make queries smarter and shorter.

1. String Patterns (`LIKE` Operator)

- **Purpose:** Used to search for a specific text pattern within a column.
- **Wildcard Character (`%`):** The percent sign (`%`) is used to substitute (replace) missing characters. It can be placed before, after, or on both sides of the text pattern.
- **Example Scenario:** We don't remember the exact name of an author, but we know it starts with the letter 'R'.
- **Query:**

```
SELECT firstname FROM author WHERE firstname LIKE 'R%';
```

- **Concept Output (Author Table):**

firstname
Raul
Rav

(Only rows where the name starts with 'R' will be returned.)

2. Ranges (`BETWEEN ... AND` Operator)

- **Purpose:** Used to filter values that fall within a specific range (inclusive of both start and end values).
- **Benefit:** It makes long conditions using `>=` (greater than or equal to) and `<=` (less than or equal to) shorter and much easier to read.
- **Example Scenario:** Retrieve books that have between 290 and 300 pages.
- **Query:**

```
-- Normal way (Long method):
-- WHERE pages >= 290 AND pages <= 300

-- Smart way (BETWEEN operator):
SELECT * FROM Book WHERE pages BETWEEN 290 AND 300;
```

3. Sets of Values (`IN` Operator)

- **Purpose:** Used to specify multiple exact values (a set of values) together in a single `WHERE` clause.
- **Benefit:** When data cannot be grouped into numeric ranges (like text values), we use `IN` to avoid writing multiple `OR` conditions over and over again.
- **Example Scenario:** We want data for authors who are from 'Australia' or 'Brazil'. If we needed to add Canada, India, and China as well, writing out `OR` conditions for each would make the query way too long.
- **Query:**

```
-- Without IN operator (Long method):
-- WHERE country = 'Australia' OR country = 'Brazil'

-- Smart way (IN operator):
SELECT * FROM author WHERE country IN ('Australia', 'Brazil');
```

---

11. Sorting Result Sets (ORDER BY)

When retrieving data, results are not automatically sorted. We use the `ORDER BY` clause to organize the output alphabetically or numerically.

1. Default Sorting (Ascending Order)

- **Rule:** By default, `ORDER BY` sorts data in Ascending order (A to Z, or lowest to highest number).
- **Example Scenario:** A list of book titles arranged alphabetically.

- **Query:**

```
SELECT title FROM book ORDER BY title;
```

- **Conceptual Output:**

title (Sorted A-Z)
A Tale of Two Cities
Harry Potter
The Great Gatsby
To Kill a Mockingbird

2. Descending Sorting (DESC Keyword)

- **Rule:** To sort data in reverse order (Z to A, or highest to lowest), add the `DESC` keyword.
- **Sorting Logic:** If the first characters match, the database checks character-by-character to determine the order.
- **Query:**

```
SELECT title FROM book ORDER BY title DESC;
```

- **Conceptual Output:**

title (Sorted Z-A)
To Kill a Mockingbird
The Great Gatsby
Harry Potter
A Tale of Two Cities

3. Sorting by Column Index

- **Rule:** Instead of typing the full column name, you can use its sequence number in the `SELECT` list.
- **Benefit:** Saves time and makes the query shorter.
- **Query:**

```
-- '2' represents the second column (pages)
SELECT title, pages FROM book ORDER BY 2;
```

- **Conceptual Output:**

title (Column 1)	pages (Column 2 - Sorted ASC)
The Great Gatsby	180
To Kill a Mockingbird	281
A Tale of Two Cities	341
Harry Potter	500

---

12. Grouping and Filtering Result Sets

Used to summarize, count, and filter grouped data (critical for answering business questions like "total sales per product").

1. Eliminating Duplicates (`DISTINCT`)

- **Rule:** Returns only unique values from a column with repeated data.
- **Query:**

```
SELECT DISTINCT country FROM author ORDER BY 1;
```

2. Grouping Data & Counting (`GROUP BY`)

- **Rule:** Groups rows with identical values into summary rows. Almost always used with aggregate functions (`COUNT()`, `SUM()`, `AVG()`).

- **Example:** How many authors come from each country?
- **Query:**

```
SELECT country, COUNT(country) FROM author GROUP BY country;
```

3. Renaming Columns (`AS` Keyword)

- **Rule:** Gives a clean, readable alias to calculated columns (instead of showing ugly default names like `COUNT(country)`).
- **Query:**

```
SELECT country, COUNT(country) AS Count FROM author GROUP BY country;
```

4. Filtering Grouped Data (`HAVING` Clause)

- **Rule:** Filters data *after* it has been grouped.
- **Query:** Show only countries with more than 4 authors.

```
SELECT country, COUNT(country) AS Count FROM author GROUP BY country
HAVING COUNT(country) > 4;
```

> 💡 **Interview Tip (WHERE vs. HAVING):** `WHERE` filters individual rows *before* grouping, while `HAVING` filters grouped totals/aggregates *after* grouping.

---

13. Built-in Database Functions

Using SQL functions directly is a best practice for processing large datasets efficiently before sending data to an application.

1. Aggregate Functions (Column Functions)

- **Rule:** Takes a collection of values (an entire column) and returns a single summarized value.

- **Common Functions:**
- `SUM()`: Adds all values.
- `MAX()` / `MIN()`: Finds highest/lowest values.
- `AVG()`: Calculates the mean.
- **Query Examples:**

```
SELECT SUM(COST) AS SUM_OF_COST FROM PETRESCUE;
SELECT MAX(QUANTITY) FROM PETRESCUE;
SELECT AVG(COST / QUANTITY) FROM PETRESCUE WHERE ANIMAL = 'Dog';
```

2. Scalar & String Functions

- **Rule:** Operates row-by-row on individual values.
- **Common Functions:**
- `ROUND()`: Rounds numbers.
- `LENGTH()`: Returns string character count.
- `UCASE()` / `LCASE()` (or `UPPER()` / `LOWER()`): Converts text casing.
- **Query Examples (Smart Filtering & Nesting):**

```
-- Lowercase filtering handles messy real-world data entries ('CAT',
'Cat', 'cat')
SELECT * FROM PETRESCUE WHERE LOWER(ANIMAL) = 'cat';

-- Nesting functions (Uppercase unique names)
SELECT DISTINCT(UPPER(ANIMAL)) FROM PETRESCUE;
```

---

14. Date and Time Functions

Crucial for generating time-series reports and dashboards in analytics.

1. Data Types

- `DATE`: 8 digits (`YYYYMMDD`)
- `TIME`: 6 digits (`HHMMSS`)
- `TIMESTAMP`: 20 digits (`YYYYXXDDHHMMSSZZZZZZ` where XX=Month, ZZZZZZ=Microseconds)

2. Date Operations

- **Extracting Parts:** Use `DAY()`, `MONTH()`, `YEAR()` to pull specific parts of a date.

```
SELECT DAY(RESCUE_DATE) FROM PETRESCUE WHERE ANIMAL = 'Cat';
```

- **Filtering by Date:**

```
SELECT COUNT(*) FROM PETRESCUE WHERE MONTH(RESCUE_DATE) = 5;
```

- **Date Arithmetic:** Adding/subtracting time intervals.

```
SELECT DATE_ADD(RESCUE_DATE, INTERVAL 3 DAY) FROM PETRESCUE;
```

- **Special Registers:** Getting the current system time.

```
SELECT CURRENT_DATE - RESCUE_DATE FROM PETRESCUE;
```

---

15. Subqueries (Nested Queries)

A query nested inside parentheses `()` within another main query. Used to bypass basic SQL limitations for complex data extraction.

1. In the `WHERE` Clause

- **Problem:** Aggregate functions (`AVG`, `MAX`) cannot be used directly in a `WHERE` clause.

- **Solution:** Use a subquery to calculate the aggregate first.
- **Query:** Retrieve employees earning less than the company average.

```
SELECT EMP_ID, FIRST_NAME, LAST_NAME, SALARY
FROM EMPLOYEES
WHERE SALARY < (SELECT AVG(SALARY) FROM EMPLOYEES);
```

2. In the Column List

- **Problem:** Comparing individual row data against an aggregate metric throws a `GROUP BY` error in standard queries.
- **Solution:** Put the subquery in the `SELECT` statement to calculate it for every row.
- **Query:**

```
SELECT EMP_ID, SALARY, (SELECT AVG(SALARY) FROM EMPLOYEES) AS AVG_SALARY
FROM EMPLOYEES;
```

3. In the `FROM` Clause (Derived Tables)

- **Concept:** Treats the subquery result as a temporary, virtual table (useful for data security or staged grouping).
- **Query:**

```
SELECT * FROM (
    SELECT EMP_ID, FIRST_NAME, LAST_NAME, DEPT_ID FROM EMPLOYEES
) AS EMP4ALL;
```

---

16. Accessing Multiple Tables (Implicit Joins)

Real industry projects require pulling data scattered across multiple tables.

1. Using Sub-queries with Multiple Tables

- **Rule:** Use the result of one table's query as a filter for another table.
- **Query:** Get employees working in a specific location by checking the departments table.

```
SELECT * FROM employees
WHERE DEP_ID IN (SELECT DEP_ID_DEP FROM departments WHERE LOC_ID =
'L0002');
```

2. Implicit Joins (Cartesian Product)

- **Warning:** Simply running `SELECT * FROM table1, table2` creates a massive Cartesian product (every row joins with every row).
- **Fixing with `WHERE` (Matching IDs):** Link them on their common Primary/Foreign Key.

```
SELECT * FROM employees, departments
WHERE employees.DEP_ID = departments.DEP_ID_DEP;
```

- **Using Table Aliases (Best Practice):** Give tables short nicknames in the `FROM` clause for cleaner code.

```
SELECT E.EMP_ID, D.DEP_NAME
FROM employees E, departments D
WHERE E.DEP_ID = D.DEP_ID_DEP;
```